

Student Name: Samrudh Patil

SRN: PES1UG24CS417

Course: Software Engineering (UE22CS351A)

Assignment Topic: Space Invaders Debugging & Enhancement

1. Deliverables & Submission Links

Deliverable	Link / Details
Before Video (10s Bug Demonstration)	https://drive.google.com/file/d/1R_lrPNGwi9eg_aFYdjUfJbezwhTbwlVu/view?usp=sharing
After Video (10s Full Features)	https://drive.google.com/file/d/1jJZSjnTkjWDD3AQW97pUZ67wQdE-Kwy0/view?usp=sharing
Assigned Starter Repository	https://github.com/SETAPESU26/46_space-invaders
Individual Lab Repository (Lab-4)	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417/tree/main/Lab-4
Prompt Evidence / Chat History	Public pair-programming interaction and prompt logs documented below in Section 4.

2. Git Commit History in Repository (Lab-4 Folder)

In accordance with the lab handout instructions, the implementation was committed in distinct, progressive task commits:

Commit Hash	Task	Description
5e2575a	Task 1: Collision Fix	Fix bullet collision detection via shallow copy iteration (<code>self.player_bullets[:]</code>)
632e242	Task 2: Game Over Screen	Implement graphical Game Over screen, dark overlay modal, and final score display
ff39973	Task 3: Replay & Difficulty	Add replay options with Easy, Medium, and Hard difficulty presets and HUD badge
ee8e464	Task 4: Sound Effects	Add retro 8-bit sound effects for shooting, enemy destroy, and game over

3. Detailed Task Implementation & Code Snippets

Task 1: Refined Bullet Collision Detection Fixed

Root Cause: The starter code looped over `for bullet in self.player_bullets:` while calling `self.player_bullets.remove(bullet)` inside the loop. In Python, modifying a list while iterating over it shifts all subsequent elements to the left by one index. Consequently, the loop's internal index counter skipped the bullet immediately following the deleted one, causing rapid-fired bullets to ghost clean through enemy aliens.

Solution: Iterate over a shallow slice copy `self.player_bullets[:]`, allowing safe in-place removals from the master list without index skipping.

```
# TASK 1: FIX BULLET COLLISION VIA SHALLOW COPY ITERATION
for bullet in self.player_bullets[:]:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            self.sound_manager.play_enemy_hit()
            break
```

Task 2: Graphical Game Over Condition & Score Screen Enhanced

Problem: Previously, losing simply paused the frame updates and printed "Game over!" to the terminal.

Solution: Implemented a graphical Pygame overlay using an alpha-blended surface (`pygame.SRCALPHA`) over the frozen game screen, featuring a centered modal card, prominent "GAME OVER" / "VICTORY!" banners, the final score, and responsive keyboard event listeners.

```
if self.game_over:
    # 1. Dark semi-transparent overlay
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 195))
    screen.blit(overlay, (0, 0))

    # 2. Centered dialog box
    card_rect = pygame.Rect(box_x, box_y, box_width, box_height)
    pygame.draw.rect(screen, (24, 24, 34), card_rect, border_radius=14)
    pygame.draw.rect(screen, accent_color, card_rect, width=3, border_radius=14)

    # 3. Title & Final Score
    title_surf = self.title_font.render(title_text, True, accent_color)
    screen.blit(title_surf, title_surf.get_rect(center=(self.width // 2, box_y + 45)))
    score_surf = self.font.render(f"Final Score: {self.score}", True, WHITE)
    screen.blit(score_surf, score_surf.get_rect(center=(self.width // 2, box_y + 95)))
```

Task 3: Replay Option & Difficulty Modes Feature Added

Implementation: Configured 3 difficulty tiers altering enemy movement speed and projectile frequency. Integrated an in-game HUD difficulty badge and dedicated hotkeys ([1] Easy, [2] Med, [3] Hard, [Q] Exit) on the Game Over screen.

```
DIFFICULTIES = {
    "Easy": {"enemy_speed": 1.0, "fire_chance": 0.001, "color": (100, 220, 100)},
    "Medium": {"enemy_speed": 1.6, "fire_chance": 0.003, "color": (240, 200, 60)},
    "Hard": {"enemy_speed": 2.5, "fire_chance": 0.007, "color": (255, 80, 80)},
}

def reset_game(self, difficulty=None):
    if difficulty and difficulty in self.DIFFICULTIES:
        self.current_difficulty = difficulty
        diff_cfg = self.DIFFICULTIES[self.current_difficulty]
        self.enemy_grid = EnemyGrid(self.width, speed=diff_cfg["enemy_speed"])
        self.enemy_fire_chance = diff_cfg["fire_chance"]
        self.player_bullets.clear()
        self.enemy_bullets.clear()
        self.score = 0
        self.game_over = False
```

Task 4: Sound Effects & Audio Feedback Feature Added

Implementation: Developed game/sound.py using standard library wave and math modules to synthesize pure retro 8-bit arcade audio directly, requiring no external sound packs:

- **Player Firing:** Downward frequency laser sweep (950 Hz down to 300 Hz) in shoot.wav
- **Enemy Destroyed:** Decaying rumble and white-noise crunch in enemy_hit.wav
- **Game Over:** 4-tone descending minor arpeggio (360 Hz → 300 Hz → 240 Hz → 180 Hz) in game_over.wav

```
class SoundManager:
    def __init__(self):
        # Initializes pygame.mixer and loads synthesized 8-bit sound effects
        self.shoot_sound = pygame.mixer.Sound("game/assets/sounds/shoot.wav")
        self.hit_sound = pygame.mixer.Sound("game/assets/sounds/enemy_hit.wav")
        self.game_over_sound = pygame.mixer.Sound("game/assets/sounds/game_over.wav")

    def play_shoot(self): self.shoot_sound.play()
    def play_enemy_hit(self): self.hit_sound.play()
    def play_game_over(self): self.game_over_sound.play()
```

4. AI / LLM Pair-Programming Evidence Summary

Phase	Prompt & Interaction Objective	Resolution
1. Setup & Debugging	Diagnose why bullets occasionally passed straight through enemy blocks.	Identified in-place list mutation during for loop; resolved using slice copy [:].
2. Before-Bug Reproduction	Automate simulation to isolate and visually demonstrate skipped collision frames.	Built generate_before_video.py

		generating 10s video of the skipped collision bug.
3. UI & State Management	Replace console print with Pygame graphical Game Over card and score.	Created translucent alpha surface overlay and input-listening modal.
4. Gameplay Tuning	Add Easy, Medium, Hard replay options and adjust enemy projectile frequency.	Integrated DIFFICULTIES dictionary, HUD indicator, and instant restart hotkeys.
5. Audio Engineering	Provide audio feedback for player firing, enemy destruction, and game over.	Synthesized standalone retro arcade .wav files with wave module.